RURAL LEDGER & MACHINERY MANAGEMENT SYSTEM

Tamil-First | Offline-First | Mobile-First

FULL SDLC PROJECT DOCUMENTATION — VERSION 2.0

Revised & Refined Edition — All Issues Resolved

Project Name	Rural Ledger & Machinery Management System
Version	2.0.0 — Revised Final
Prepared For	Rural Entrepreneur Client — Tamil Nadu, India
Document Type	Full SDLC Documentation (Phase 1 – Phase 6)
Revision	All critical issues resolved; 3 new modules added
Date	April 2026
Classification	Client Delivery Document

This is the authoritative and final specification for the system. All development must follow this document.

PHASE 1

Project Charter & Planning Document

1.1 Project Overview

The Rural Ledger & Machinery Management System is a purpose-built digital platform for a Tamil Nadu-based rural entrepreneur who manages a harvester hiring service, tractor and machinery rentals, own-land farming, worker management, and informal lending. The system replaces fragmented, error-prone notebook tracking with a structured, reliable, and easy-to-use digital solution.

Three foundational priorities drive every design decision: Tamil-language-first UI for native-speaking users, offline-first data storage for areas with limited internet, and mobile-first design to match the primary devices used in the field.

1.2 Problem Statement

The client currently manages all business records using handwritten notebooks. This method creates the following documented problems:

- Frequent arithmetic errors in income, expense, and profit calculations across harvester, rental, and farming activities.
- No consolidated daily, weekly, or monthly profitability view across all business lines.
- Diesel usage and machine hour tracking are inaccurate — sessions noted informally, totalled manually.
- Driver salary calculation (based on machine hours) has no automated support — done mentally or on paper.
- Worker salary, advance, and bonus records are scattered across multiple notebooks and prone to disputes.
- Farmer-wise billing and payment history is unavailable, making it impossible to verify amounts or track partial payments over time.
- The admin cannot generate reports or summaries without manually adding up notebook entries.
- No unique bill reference numbers exist, so billing disputes cannot be traced.

1.3 Solution Overview

A modular single-page web application (SPA) built on React with Vite. Data is persisted in browser LocalStorage. The system has 12 modules covering all business activities. All calculations are automated, data is stored instantly, and the interface uses Tamil labels throughout. A Bill ID system provides unique traceable references for all harvester billing. A Driver Salary Ledger automates salary calculation from machine log hours.

1.4 Stakeholder Analysis

Stakeholder	Role	Primary Interest	Interaction Level
Business Entrepreneur	Admin / Owner	Full control, accurate reports, driver salary	Daily — Full access
Machine Driver	Driver	Quick log entry, view own salary	Daily — Log entry + salary view
Village Farmer	Farmer	View billing and payment history	On-demand — Read only
Workers	Indirect	Accurate salary records	None (admin manages)

1.5 User Role Definitions

Admin (Entrepreneur)

Full unrestricted access to all 12 modules. Manages pricing configuration, user accounts, reports, and export. Uses mobile in the field and laptop at home. Creates and manages Driver and Farmer accounts including driver salary rates.

Driver

Field operator on mobile. Enters machine usage logs (machine, farmer, sessions, diesel). Can also view their own salary ledger — what they have earned and what has been paid. Drivers do not access financial data, pricing, or other farmers' records. Accounts and salary rates are Admin-managed.

Farmer (Read-Only)

Views personal transaction history — machine hours, bills (with Bill IDs), payment history, and balance due. No data entry. Identified by name or ID with no password required. Interface is Tamil-first and intentionally minimal.

1.6 Operating Environment & Constraints

Constraint	Description	Design Response
Internet	Unstable or absent in rural areas	Fully offline-first; zero API dependency
Device	Primarily Android mobile phones	Mobile-first responsive layout, 44px+ touch targets
Literacy	Some users have limited reading literacy	Tamil labels, icon-assisted UI, minimal navigation
Language	Tamil is the primary	Every field has a Tamil label

	language	
Technical Skill	Non-technical users	Large buttons, simple forms, confirmation messages
Data Entry Timing	Logs entered during or after work	Multi-session support, no real-time requirement

1.7 Project Scope

In Scope — 12 Modules

- M01 Dashboard — Aggregated income, expense, and profit summary
- M02 User Management — CRUD for Farmers, Drivers, Workers
- M03 Finance Ledger — Household expense and informal lending
- M04 Own Farming — Personal agriculture expense and harvest income
- M05 Machinery Rental — Tractor, plough, trailer, water rental billing
- M06 Harvester Management — Session billing, expenses, Bill ID, per-job profit
- M07 Worker Management — Work entries, salary, advance, net payable
- M08 Driver Logs — Multi-session time entry, diesel tracking
- M09 Farmer View — Read-only personal transaction and payment portal
- M10 Driver Salary Ledger — Hours-based salary, partial payment, monthly balance
- M11 Reports — Monthly, machine-wise, farmer balance, and driver salary reports
- M12 Pricing Configuration — Admin-managed rates for all machine types

Out of Scope (Future Phases)

- Cloud sync, multi-device real-time sync
- Online payment integration, banking API
- Multi-entrepreneur / multi-tenant support
- OTP login, advanced analytics

1.8 Project Timeline

Phase / Day	Focus Area	Deliverable
Day 1	Project setup, base structure, pricing config	Running app with folder layout, components, pricing init
Day 2	Driver entry UI	Complete visual form, Tamil labels, mobile layout
Day 3	Time calculation logic	Accurate session-based hour calculations with overnight guard
Day 4	Diesel logic + data save	Data persists via LocalStorage; diesel both modes working
Day 5	Harvester module + Bill ID	Rent calculation, Bill ID generation, job linking

Day 6	Machinery rental + payment history	Rental entries with payments[] array
Day 7	Worker management + Driver salary ledger	Worker net-pay; driver salary from log hours
Day 8	Dashboard	Live income/expense/profit from all 12 modules
Day 9	Reports + PWA setup	All report types; installable offline app
Day 10	Export, testing, polish	PDF, backup/restore with schemaVersion, end-to-end UAT

1.9 Risk Register

Risk	Probability	Impact	Mitigation
Browser storage cleared	Medium	High	JSON export with schemaVersion; weekly backup habit
Low usability	Medium	High	Tamil UI, large elements, usability testing pre-delivery
No internet	High	High	Full offline-first PWA architecture
Calculation errors	Low	High	Unit test every calculation function before integration
Old Android device	Medium	Medium	Test on low-end device before delivery
Scope creep	Medium	Medium	Strict MVP; no unapproved additions during build

1.10 Success Criteria

1. Driver logs a full entry (sessions + diesel) without assistance in under 2 minutes.
2. Harvester rent auto-calculated from logged hours; Bill ID generated automatically.
3. Driver salary auto-calculated from machine hours; partial payments tracked.
4. All data survives app close and reopen with WiFi off.
5. Admin views monthly income, expense, and profit in under 10 seconds.
6. Farmer views their bills and payment history by name or ID alone.
7. JSON backup exports, then restores with full data integrity and schemaVersion validation.
8. App passes usability test with real driver and real farmer in village conditions.

PHASE 2 Software Requirements Specification (SRS)

2.1 Purpose of This Document

This SRS defines, with complete precision, the functional and non-functional requirements for all 12 modules of the system. Every requirement is traceable to a business need in Phase 1. This document is the binding specification — no behaviour should be implemented that is not defined here, and no defined behaviour should be omitted.

2.2 System Modules Overview

Module ID	Module Name	Primary User	Core Purpose
M01	Dashboard	Admin	Real-time aggregated summary from all modules
M02	User Management	Admin	CRUD for Farmers, Drivers, Workers
M03	Finance Ledger	Admin	Household expenses and informal lending
M04	Own Farming	Admin	Personal agriculture expense and income
M05	Machinery Rental	Admin	Tractor, plough, trailer, water rental billing
M06	Harvester Management	Admin	Harvester billing, Bill ID, per-job expenses and profit
M07	Worker Management	Admin	Worker records, salary, advance, net payable
M08	Driver Logs	Driver / Admin	Machine session logs, diesel tracking
M09	Farmer View	Farmer	Read-only bills, payment history, balance
M10	Driver Salary Ledger	Admin / Driver	Hours-based salary, partial payments, monthly balance
M11	Reports	Admin	Monthly, machine-wise, farmer balance, driver salary
M12	Pricing Configuration	Admin	All machine rates, driver salary rates, diesel price

2.3 Authentication Requirements

Role	Login Method	Password	Access Granted
Admin	Single password screen	Set during first launch	All 12 modules — full CRUD
Driver	Select name from list + 4-	Set by Admin	Driver Logs (entry) + own

	digit PIN		Salary Ledger (read)
Farmer	Select name from list	None required	Own records, bills, payment history — read only

2.4 Module-Wise Functional Requirements

M01 — Dashboard Module

Display Element	Data Source	Calculation
Total Income (Month)	Harvester + Rental + Own Farm Harvest	Sum all income records for current month
Total Business Expense (Month)	Expenses (source != home_expense, lending)	Sum business expense records for current month
Net Profit (Month)	Derived	Total Income minus Total Business Expense
Today's Machine Sessions	Driver Logs	Count of log entries for today
Today's Diesel Cost	Driver Logs	Sum of diesel.total from today's logs
Pending Harvester Bills	Harvester Jobs	Count where paymentStatus = pending or partial
Pending Farmer Balance	Harvester + Rental	Sum of all balance fields > 0
Pending Lending Amount	Finance Ledger	Sum of (amountGiven - amountReturned) where status != returned
Driver Salary Unpaid	Driver Salary Ledger	Sum of unpaid balances across all drivers

M02 — User Management Module

Admin-only. Provides full CRUD for Farmers, Drivers, and Workers. Deletion rules (see Section 2.6) prevent orphaned reference data.

Farmer Entity

Field	Type	Validation	Tamil Label
id	String (auto)	Format F001, F002... F000 reserved for Own Farm	—
name	String	Required, max 60 chars	விவசாயி பெயர்
village	String	Required	கிராமம்

phone	String	Optional, 10 digits	தொலைபேசி
type	Enum	neighbour | normal | own (F000 only)	வகை

Driver Entity

Field	Type	Validation	Tamil Label
id	String (auto)	Format D001, D002...	—
name	String	Required, max 60 chars	ஓட்டுநர் பெயர்
phone	String	Optional, 10 digits	தொலைபேசி
pin	String	4-digit numeric, Admin-set	கடவுச்சொல்
salaryRatePer Hour	Number	Required > 0; used in Driver Salary Ledger	மணி நேர ஊதிய விகிதம் (₹)

Worker Entity

Field	Type	Validation	Tamil Label
id	String (auto)	Format W001, W002...	—
name	String	Required, max 60 chars	தொழிலாளர் பெயர்
village	String	Optional	கிராமம்
phone	String	Optional, 10 digits	தொலைபேசி
active	Boolean	Default true; set false to deactivate (not delete)	நிலை

M03 — Finance Ledger Module

Home Expense Sub-Module

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
date	Date	Required, not future	தேதி
category	Enum	Food | Medical | Transport | Utilities | Other	வகை
mealTime	Enum	morning | afternoon | evening | null	உணவு நேரம் (food only)
description	String	Optional, max 100 chars	விளக்கம்

amount	Number	Required, > 0	தொகை (₹)

mealTime field: Applicable only when category = Food. Allows separate morning, afternoon, and evening food entries on the same date to be distinguished in reports and storage. For all other categories, mealTime is null.

Lending Sub-Module

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
personName	String	Required	பெயர்
village	String	Optional	கிராமம்
amountGiven	Number	Required, > 0	கொடுத்த தொகை (₹)
dateLent	Date	Required	கொடுத்த தேதி
amountReturned	Number	Computed from repayments[]	திரும்ப பெற்ற தொகை (₹)
repayments	Array	Each: { id, date, amount, notes }	திரும்பிய பதிவுகள்
status	Enum	pending | partial | returned	நிலை
notes	String	Optional	குறிப்பு

Lending Repayments and Dashboard Income: Amounts lent and amounts repaid are NOT counted as business income or expense in the Dashboard profit calculation. They are tracked in the Finance Ledger only. The Dashboard shows 'Pending Lending Amount' as a separate awareness indicator, not a profit component. This prevents lending activity from inflating or deflating business profit figures.

M04 — Own Farming Module

Tracks the admin's personal agriculture. Separate from harvester business. Machine hours for own land are logged using reserved Farmer ID F000 in Driver Logs — this prevents double-counting while keeping all machine activity in one unified log.

Own Farming Expense Fields

Category	Input	Special Rule	Tamil Label
Diesel	Amount (₹)	Separate from machine diesel in Driver Logs (see Section 2.5)	டீசல்

Maintenance	Amount (₹)	Machine maintenance	பராமரிப்பு
Spare Parts	Amount (₹)	Replacement parts	உதிரி பாகங்கள்
Driver Salary	Amount (₹)	Manual entry if not using Driver Salary Ledger	ஓட்டுநர் சம்பளம்
Worker Salary	Amount (₹)	Field worker payments	தொழிலாளர் சம்பளம்
Food Expense	Amount (₹)	Multiple per day: Morning / Afternoon / Evening (mealTime field)	உணவு செலவு
Motor Maintenance	Amount (₹)	Irrigation motor	மோட்டார் பராமரிப்பு

Own Farming Income Fields

Field	Type	Calculation	Tamil Label
Harvest Date	Date	Required	அறுவடை தேதி
Crop Type	String	e.g., Paddy, Sugarcane	பயிர் வகை
Number of Bags	Number	Required, > 0	மூட்டைகளின் எண்ணிக்கை
Price Per Bag (₹)	Number	Required, > 0	ஒரு மூட்டை விலை
Total Income (auto)	Computed	Bags × Price Per Bag	மொத்த வருமானம்

M05 — Machinery Rental Module

Machine	Unit	Rate Source	Tamil Name
Tractor	Per Hour	Pricing Config (M12)	டிராக்டர்
Plough Tractor	Per Hour	Pricing Config (M12)	உழவு டிராக்டர்
Trailer	Per Trip	Pricing Config (M12)	ட்ரெய்லர்
Water Rental	Per Season	Admin assigns per farmer	தண்ணீர் குத்தகை

Rental Entry Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
date	Date	Required	தேதி

farmerId	Reference	Must exist in Farmers	விவசாயி
machineType	Enum	tractor | plough_tractor | trailer | water	இயந்திர வகை
unit	Enum	hour | trip | season	அலகு
quantity	Number	Required, > 0	அளவு
ratePerUnit	Number	Fetched from Pricing Config, admin-editable per entry	ஒரு அலகு விலை
totalAmount	Computed	quantity × ratePerUnit	மொத்த தொகை
payments	Array	Each payment: { id, date, amount, mode, notes }	பணம் செலுத்திய வரலாறு
amountPaid	Computed	Sum of payments[].amount	மொத்தம் செலுத்தியது
balance	Computed	totalAmount − amountPaid	நிலுவை தொகை
paymentStatus	Computed	Auto-derived: pending | partial | paid	பணம் நிலை

M06 — Harvester Management Module

The core revenue module. Links to Driver Logs for automatic hour computation, generates a unique Bill ID for every job, and tracks per-job expenses and profitability.

Bill ID System

```
Format:  HB-SSS-YYYY-NNN

HB   = Harvester Bill (fixed prefix)
SSS  = Season code: KUR (Kuruvai), SAM (Samba), THA (Thaladi)
YYYY = 4-digit year of the season
NNN  = 3-digit running number within that SSS-YYYY combination, starting at 001

Examples:
  HB-KUR-2026-001  First Kuruvai 2026 harvester bill
  HB-KUR-2026-002  Second Kuruvai 2026 harvester bill
  HB-SAM-2026-001  First Samba 2026 harvester bill

Season Definitions:
  KUR — Kuruvai: June to September
  SAM — Samba:   August to January (crosses year — use start year)
  THA — Thaladi: October to January (crosses year — use start year)
```

Generation Rule: On saving a new Harvester Job, count existing jobs with the same SSS-YYYY combination, add 1, zero-pad to 3 digits.

Diesel Separation Rule (Resolves Double-Counting)

Two types of diesel exist in the harvester workflow — they must NEVER be mixed:

Type 1 — Session Diesel (in Driver Logs):
Entered by the driver per work session. Represents fuel physically added to the machine on that day. This is the operational diesel cost.

Type 2 — Additional Job Diesel (in Harvester Job expenses):
Entered by the Admin for diesel costs NOT captured in driver logs.
Example: bulk diesel bought at the start of a season, transport diesel, or diesel for a session where the driver forgot to log it.

In the Harvester Job record:
dieselFromLogs = auto-computed sum of diesel.total from all linkedLogIds (read-only)
additionalDiesel = manual Admin entry for extra diesel not in logs (default 0)
totalDiesel = dieselFromLogs + additionalDiesel

Dashboard expense uses totalDiesel per job. Never counts dieselFromLogs twice.

Harvester Log-Linking UX Rules

Rule	Description
Filter scope	When linking logs, show only logs matching this job's farmerId + machineType
Date range filter	Optional date range picker to narrow down logs for long seasons
Already-linked guard	Logs already linked to another Harvester Job are shown greyed out and non-selectable
Duplicate guard	Same log cannot be linked to two active jobs — system prevents this on save
Unlink rule	Admin can unlink a log only if the Harvester Job has zero payments recorded
Auto-compute	On each checkbox change, totalHours recalculates instantly in the UI

Harvester Job Record Fields

Field	Type	Validation / Notes	Tamil Label
id	String (auto)	Auto-generated internal ID	—

billId	String (auto)	HB-SSS-YYYY-NNN format, generated on save	பில் எண்
farmerId	Reference	Must exist in Farmers	விவசாயி
village	String	Auto-populated from farmer record	கிராமம்
season	Enum	KUR | SAM | THA	பருவம்
seasonYear	Number	4-digit year of season start	ஆண்டு
machineType	Enum	tyre | track	இயந்திர வகை
wetField	Boolean	Applicable if machineType = tyre only	நீர் நிலம்
ratePerHour	Number	Fetched from Pricing Config; Admin-editable	மணி நேர விலை
linkedLogIds	Array	IDs of linked Driver Log records	—
totalHours	Computed	Sum of hours from linked Driver Logs	மொத்த மணி நேரம்
grossAmount	Computed	totalHours × ratePerHour	மொத்த வாடகை
discount	Number	Default 0	தள்ளுபடி
finalAmount	Computed	grossAmount − discount	இறுதி தொகை
dieselFromLogs	Computed	Sum of diesel.total from linkedLogIds (read-only)	பதிவிலிருந்து டீசல் (₹)
additionalDiesel	Number	Default 0, manual Admin entry	கூடுதல் டீசல் (₹)
otherExpenses	Array	[{ category, amount }] — worker, mechanic, parts, food	மற்ற செலவுகள்
totalExpense	Computed	dieselFromLogs + additionalDiesel + sum(otherExpenses)	மொத்த செலவு
netProfit	Computed	finalAmount − totalExpense	நிகர லாபம்
payments	Array	[{ id, date, amount, mode, notes }]	பணம் செலுத்திய வரலாறு
amountPaid	Computed	Sum of payments[].amount	மொத்தம் செலுத்தியது
balance	Computed	finalAmount − amountPaid	நிலுவை தொகை
paymentStatus	Computed	pending | partial | paid (auto-derived)	பணம் நிலை

M07 — Worker Management Module

Worker Net Payable Formula

Net Payable = (Base Salary + Bonus + Extra Amount) − Advance Paid

All five fields are independently editable. Advance is subtracted because it was given to the worker before the salary payment. Net Payable is always computed — never entered manually. If Net Payable is negative, a warning is shown: the worker has received more in advance than earned, and the difference carries forward to the next entry.

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated	—
workerId	Reference	Must exist in Workers	தொழிலாளர்
date	Date	Required	தேதி
workType	Enum + Free-text	Dropdown; last option is 'Other' (manual text)	வேலை வகை
baseSalary	Number	Required, >= 0	அடிப்படை சம்பளம் (₹)
advance	Number	Default 0	முன்பணம் (₹)
bonus	Number	Default 0	போனஸ் (₹)
extraAmount	Number	Default 0	கூடுதல் தொகை (₹)
netPayable	Computed	(baseSalary + bonus + extraAmount) − advance	நிகர செலுத்தவேண்டியது (₹)
notes	String	Optional	குறிப்பு

M08 — Driver Logs Module

Foundation data-entry module. Must be the fastest and most reliable screen in the system. Its data feeds M06 Harvester, M10 Driver Salary, and M01 Dashboard.

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated, format LOG001...	—
driverId	Reference	Must exist in Drivers	ஓட்டுநர்
farmerId	Reference	Must exist (F000 valid for Own Farm)	விவசாயி
machineType	Enum	harvester_tyre | harvester_track | tractor | plough_tractor | trailer	இயந்திர வகை

date	Date	Required, defaults to today	தேதி
sessions	Array	Minimum 1; each: { start, end, durationHours }	வேலை நேரம்
totalHours	Computed	Sum of session durationHours	மொத்த மணி நேரம்
diesel.mode	Enum	rupees | litres | none	டீசல் முறை
diesel.value	Number	Required if mode != none; > 0	டீசல் அளவு
diesel.pricePerLitre	Number	Required if mode = litres; fetched from Pricing Config	லிட்டர் விலை
diesel.total	Computed	If rupees: value. If litres: value × pricePerLitre. If none: 0	டீசல் செலவு (₹)
source	Enum	driver_log (fixed)	—

Session overnight guard: If end time is numerically before start time (e.g., 23:00 to 01:00), 24 hours is added to the end time before calculating duration. This handles late-night sessions without error.
Formula: duration = (end < start) ? (end + 24h) − start : end − start

M09 — Farmer View Module

Read-only portal. Farmer selects their name from a list — no password needed. Shows all activity linked to their Farmer ID across all modules, including the full payment history per bill.

Section	Data Shown	Source
Harvester Bills	Bill ID, season, machine type, hours, final amount	M06 Harvester Jobs
Bill Payments	Each payment: date, amount, mode — for every bill	M06 payments[]
Bill Balance	Amount still owed per bill	M06 balance (computed)
Rental Charges	Machine, date, hours/trips, amount	M05 Rental Entries
Rental Payments	Each payment with date and amount	M05 payments[]
Rental Balance	Amount still owed	M05 balance (computed)
Total Outstanding	Sum of all balances across all bills and rentals	Computed on screen

M10 — Driver Salary Ledger Module

Automates driver salary calculation from machine hours already recorded in Driver Logs. This replaces manual notebook-based driver pay tracking.

Driver Salary Formula

Daily Salary Earned = totalHours (from Driver Logs that day) × salaryRatePerHour (from Driver record)

Each day a driver works, a salary entry is auto-generated from that day's logs.
Admin reviews and confirms. Partial payments are recorded with dates.

Example:
Driver Kumar — salaryRatePerHour: ₹150
April 20 logs: 4.25 hours
Salary earned: 4.25 × 150 = ₹637.50

Admin pays ₹400 on April 20:
Earned: +₹637.50
Paid: −₹400.00
Balance: ₹237.50 (carries to next payment)

Monthly view: Total earned − Total paid = Monthly outstanding per driver

Driver Salary Entry Fields

Field	Type	Validation	Tamil Label
id	String (auto)	Auto-generated DSL001, DSL002...	—
driverId	Reference	Must exist in Drivers	ஓட்டுநர்
date	Date	Required	தேதி
logIds	Array	Driver Log IDs from this date that contributed hours	—
hoursWorked	Computed	Sum of totalHours from logIds	வேலை மணி நேரம்
ratePerHour	Snapshot	Copied from Driver record at time of entry	மணி நேர விகிதம் (₹)
salaryEarned	Computed	hoursWorked × ratePerHour	சம்பாதித்த சம்பளம் (₹)
payments	Array	[{ id, date, amount, mode, notes }]	செலுத்திய வரலாறு
amountPaid	Computed	Sum of	செலுத்திய மொத்தம் (₹)

		payments[].amount	
balance	Computed	salaryEarned − amountPaid	நிலுவை (₹)

Driver view access: When a driver logs in with their PIN, they can view their own Driver Salary Ledger — earned amount, payments received, and current balance. They cannot edit any data.

M11 — Reports Module

Report 1 — Monthly Business Summary

- Filter: Select month and year
- Shows: Total income by source (harvester, rental, own farm), Total expense by source, Net profit
- Secondary: Bar or list breakdown of income sources

Report 2 — Machine-Wise Report

- Groups all harvester jobs by machineType (tyre / track)
- Per machine: Total hours, Total revenue, Total expense, Net profit
- Filter: by season (KUR / SAM / THA) and year

Report 3 — Farmer Balance Report

- Lists all farmers with outstanding balance > 0 across harvester and rental
- Per farmer: Bill IDs, original amount, amount paid, current balance
- Sorted by balance (highest first)

Report 4 — Driver Salary Report

- Filter: Select driver and month
- Shows: Each day's hours, salary earned, payments made, running balance
- Monthly total: Total earned, total paid, closing balance

Report 5 — Harvester Season Report

- Filter: Select season (KUR / SAM / THA) and year
- Lists all jobs with Bill ID, farmer name, hours, final amount, payment status
- Season totals: Total billed, total collected, total outstanding

M12 — Pricing Configuration Module

Admin-only. A single configuration screen that stores all rate information. All modules fetch rates from here — no hardcoded values exist anywhere else in the system.

Config Category	Field	Type	Tamil Label
Harvester — Tyre	Standard Rate Per Hour	Number (₹)	டயர் — சாதாரண விகிதம்
Harvester — Tyre	Wet Field Rate Per Hour	Number (₹)	டயர் — நீர் நிலம் விகிதம்
Harvester — Track	Fixed Rate Per Hour	Number (₹)	டிராக் — விகிதம்
Rental — Tractor	Rate Per Hour	Number (₹)	டிராக்டர் விகிதம்
Rental — Plough Tractor	Rate Per Hour	Number (₹)	உழவு டிராக்டர் விகிதம்
Rental — Trailer	Rate Per Trip	Number (₹)	ட்ரெய்லர் விகிதம்
Diesel	Current Price Per Litre	Number (₹)	டீசல் விலை / லிட்டர்

Water rental rate is per-farmer and per-season. It is not stored in the Pricing Config. Instead, it is entered directly in the Rental Entry for each farmer.

Driver salary rates are stored in the individual Driver record (salaryRatePerHour field in M02), not in the central Pricing Config.

2.5 Business Logic Rules

Bill ID Generation Logic

- On creating a new Harvester Job, the system reads the selected season (KUR/SAM/THA) and seasonYear.
- It counts all existing Harvester Jobs with the same season+year combination.
- New NNN = count + 1, zero-padded to 3 digits.
- Bill ID is assigned on first save. It cannot be changed after assignment.
- If a Harvester Job is deleted (cancelled), its Bill ID is never reused — NNN continues from the highest existing number.

Harvester Billing Logic (Complete Flow)

9. Admin selects farmer, season (KUR/SAM/THA), year, and machine type.
10. Admin links Driver Logs for this job using the log-linking section (filtered by farmerId + machineType).
11. System auto-computes totalHours from linked logs.
12. ratePerHour is fetched from Pricing Config based on machineType and wetField toggle.

13. grossAmount = totalHours × ratePerHour.
14. Admin optionally enters discount.
15. finalAmount = grossAmount − discount.
16. dieselFromLogs is auto-summed from linked Driver Logs (read-only).
17. Admin enters additionalDiesel if any extra diesel not in logs.
18. Admin enters otherExpenses (worker salary, mechanic, spare parts, driver food).
19. totalExpense = dieselFromLogs + additionalDiesel + sum(otherExpenses).
20. netProfit = finalAmount − totalExpense.
21. On save, Bill ID is generated (HB-SSS-YYYY-NNN format).
22. Payments are recorded in the payments[] array as they arrive.

Payment History System

Every billing record (Harvester Jobs and Rental Entries) uses a payments[] array.
Each element: { id: 'PAY001', date: '2026-04-15', amount: 5000, mode: 'cash', notes: '' }

amountPaid = payments.reduce((sum, p) => sum + p.amount, 0) [computed, never stored]
balance = finalAmount − amountPaid [computed, never stored]
paymentStatus is auto-derived:
amountPaid == 0 → 'pending'
amountPaid > 0 && balance > 0 → 'partial'
balance == 0 → 'paid'

Payment records cannot be deleted — only a correction entry can be added.
This ensures an auditable payment trail for every bill.

Diesel Rules (No Double-Counting)

Scenario	Correct Action	Wrong Action (Do NOT Do This)
Driver logs 10L diesel for a session	Driver enters in Driver Logs. dieselFromLogs auto-sums.	Admin also enters 10L in Harvester Job additionalDiesel
Admin buys bulk diesel not in logs	Admin enters in Harvester Job additionalDiesel	Admin asks driver to retroactively create a log entry
Own farm diesel	Admin enters in Own Farming expenses (source: own_farm)	Admin enters in a Harvester Job expense

Edit and Delete Rules

Record Type	Edit Rule	Delete Rule
Driver Log	Admin only. If log is linked to a Harvester Job, editing hours triggers recalculation of that job's	Cannot delete if linked to a Harvester Job with payments. Admin must first remove the link or cancel the job.

	totalHours and grossAmount automatically.	
Harvester Job	Admin only. Full editing allowed. Changing linked logs or rate auto-recomputes all derived fields.	Soft-cancel only (set status = cancelled). Bill ID is retired, not reused. Cannot hard-delete if payments exist.
Rental Entry	Admin only. Can edit if no payments recorded. If payments exist, only notes and paymentStatus editable.	Cannot delete if any payment recorded. Set status = cancelled instead.
Worker Entry	Admin only. Fully editable within 7 days of creation.	Can delete only if created today and no external references exist.
Driver Salary Entry	Admin only. Can edit ratePerHour snapshot and notes.	Cannot delete if payment recorded against it.
Farmer Record	Admin only. Name and village editable anytime.	Cannot delete if any linked records (logs, jobs, rentals) exist. Set type to inactive instead.
Driver Record	Admin only. Name, PIN, and ratePerHour editable.	Cannot delete if any Driver Logs exist. Deactivate instead.
Payment Record	Cannot be edited.	Cannot be deleted. Add a correction entry (negative amount with note) if error occurred.

2.6 Validation Rules

Rule	Condition	Error / Behaviour
Session end time	End must be after start (after overnight guard)	Error shown inline; entry not saved
Diesel value	Must be > 0 if mode is not 'none'	Error shown
Price per litre	Required and > 0 if mode = litres	Error shown
Farmer reference	farmerId must exist in Farmers collection	Error shown; blocked on save
Required fields	All fields marked required must be non-empty	Error per field
Net payable (worker)	May be negative — carry-forward situation	Warning shown; save allowed
Driver log linking	Same log cannot be in two active Harvester Jobs	Error shown on save attempt
Bill ID uniqueness	Auto-generated; system verifies uniqueness before saving	Auto-increments if collision detected

Monetary amounts	All monetary fields >= 0	Error shown for negative input
Season year	Must be 4-digit year, not future by more than 1	Error shown

2.7 Non-Functional Requirements

Category	Requirement	Acceptance Threshold
Performance	Page/screen load time	Under 2 seconds on 2-year-old Android phone
Offline	All features work without internet	Zero network dependency for read/write
Reliability	No data loss on app close/reopen	100% data persistence via LocalStorage
Usability	Driver logs entry without help	Achievable in under 2 minutes first time
Accessibility	Tamil labels on all input fields	Every field has a Tamil label
Compatibility	Android Chrome (primary)	Chrome 90+; desktop Chrome secondary
Maintainability	Modular code structure	Each module in its own folder; logic in services/
Data Safety	Schema versioning on backup	schemaVersion field in every JSON export

PHASE 3 System Design Document (SDD)

3.1 Architecture Overview

Single-Page Application on React + Vite. No backend server, no REST API, no database server. All data lives in browser LocalStorage as named JSON collections. The app is entirely self-contained — deployable as a static site to Vercel and fully functional offline via PWA service worker after first load.

Architecture Stack: React 18 (Vite) → Pages → Components → Services → LocalStorage

Future upgrade path: Replace storage.js calls with API calls — zero changes needed in components.
The service layer acts as the complete abstraction boundary between UI and data.

3.2 Entity Relationship Overview

The following table defines the relationships between the core data entities. Understanding these relationships is critical for building correct delete guards and cascading logic.

Entity	References	Referenced By	Cascade Rule
Farmer (F000 Own Farm)	—	Driver Logs, Harvester Jobs, Rental Entries	Cannot delete if referenced records exist
Driver	—	Driver Logs, Driver Salary Entries	Cannot delete if logs exist; deactivate instead
Worker	—	Work Entries	Cannot delete if entries exist; deactivate instead
Driver Log	Farmer, Driver	Harvester Job (linkedLogIds), Driver Salary Entry	Cannot delete if linked to a job with payments
Harvester Job	Farmer, Driver Logs	Payments (embedded)	Soft-cancel only; Bill ID retired
Rental Entry	Farmer	Payments (embedded)	Soft-cancel if payments exist
Driver Salary Entry	Driver, Driver Logs	Payments (embedded)	Cannot delete if payments recorded
Pricing Config	—	Harvester Jobs, Rental Entries, Driver Logs (diesel price)	Changing config does not affect saved records

3.3 Folder Structure

```
src/
  components/
    common/
      Button.jsx           — Reusable styled button
      InputField.jsx       — Label (Tamil+English) + input
      TamilLabel.jsx       — Dual-language label renderer
      SelectField.jsx      — Dropdown with Tamil labels
      ConfirmDialog.jsx    — Delete / cancel confirmation modal
      Badge.jsx            — Status: paid / partial / pending
      PaymentHistory.jsx   — Reusable payment list + add payment form
    dashboard/
      SummaryCard.jsx      — Single stat display card
    driver/
      SessionEntry.jsx     — Single time session row
      SessionList.jsx      — All sessions for one log
      DieselInput.jsx      — Rupees / Litres / None toggle
    harvester/
      LogLinker.jsx        — Checkbox list for linking driver logs
      RateDisplay.jsx      — Shows computed gross / final amounts
      ExpenseForm.jsx      — Additional diesel + other expenses
    rental/
      RentalForm.jsx
    worker/
      WorkerForm.jsx
      WorkTypeDropdown.jsx
    salary/
      SalaryCard.jsx       — Daily earned + paid + balance per driver
      SalaryPayForm.jsx    — Add partial payment to salary entry
    farmer/
      FarmerBillCard.jsx   — Bill ID + hours + amount + payment history
  pages/
    Login.jsx
    Dashboard.jsx
    DriverEntry.jsx
    Harvester.jsx
    Rental.jsx
    Workers.jsx
    DriverSalary.jsx
    FarmerView.jsx
    Finance.jsx
    OwnFarming.jsx
    UserManagement.jsx
    PricingConfig.jsx
    Reports.jsx
  services/
    storage.js             — All LocalStorage read / write / update / delete
    calculations.js        — All business logic: hours, rent, salary, Bill ID
    billId.js              — Bill ID generation and validation
  utils/
    timeUtils.js           — Time parsing and duration with overnight guard
    idGenerator.js         — Auto ID generation (F001, LOG001, DSL001...)
    formatters.js          — Currency, date, Tamil number display helpers
  data/
    initialData.js         — Seed: F000 Own Farm, sample pricing config
    workTypes.js           — Worker work type dropdown list
```

```
    machineTypes.js        — Machine enum labels (Tamil + English)
    seasons.js             — Season enum: KUR/SAM/THA with Tamil names
  App.jsx
  main.jsx
```

3.4 Data Schemas (Complete)

All collections are stored in LocalStorage under the keys shown. IDs are auto-generated. Collections are plain JSON arrays. The pricingConfig collection is a single JSON object (not an array).

Collection: farmers (key: 'rl_farmers')

```
{
  id:      'F000',          // F000 reserved for Own Farm
  name:    'Own Farm',      // சொந்த நிலம்
  village: '',
  phone:   '',
  type:    'own'            // own | neighbour | normal
}
```

Collection: drivers (key: 'rl_drivers')

```
{
  id:                'D001',
  name:              'Kumar',
  phone:             '9876543210',
  pin:               '1234',
  salaryRatePerHour: 150,    // ₹ per machine hour — used in Driver Salary
  active:            true
}
```

Collection: driverLogs (key: 'rl_driver_logs')

```
{
  id:          'LOG001',
  driverId:    'D001',
  farmerId:    'F001',          // 'F000' for own farm
  machineType: 'harvester_tyre',
  date:        '2026-04-20',
  sessions: [
    { start: '09:00', end: '10:45', durationHours: 1.75 },
    { start: '11:30', end: '14:00', durationHours: 2.50 }
  ],
  totalHours: 4.25,
  diesel: {
    mode:          'litres',    // 'rupees' | 'litres' | 'none'
    value:         10,
```

```
    pricePerLitre: 102,         // snapshot from Pricing Config at time of entry
    total:         1020         // computed on save; never re-computed after
  },
  source: 'driver_log'
}
```

Collection: harvesterJobs (key: 'rl_harvester_jobs')

```
{
  id:              'HJ001',
  billId:          'HB-KUR-2026-001',   // generated on save; immutable
  farmerId:        'F001',
  village:         'Kaveripattinam',
  season:          'KUR',                // KUR | SAM | THA
  seasonYear:      2026,
  machineType:     'tyre',               // 'tyre' | 'track'
  wetField:        false,
  ratePerHour:     2300,
  linkedLogIds:    ['LOG001', 'LOG002'],
  totalHours:      7.75,                 // computed from linked logs
  grossAmount:     17825,                // totalHours x ratePerHour
  discount:        500,
  finalAmount:     17325,                // grossAmount - discount
  dieselFromLogs:  2040,                 // auto-sum from linked logs (read-only)
  additionalDiesel: 500,                 // extra diesel not in logs
  otherExpenses: [
    { category: 'worker',   amount: 1500 },
    { category: 'mechanic', amount: 800  },
  ],
  totalExpense:    4840,    // dieselFromLogs + additionalDiesel +
sum(otherExpenses)
  netProfit:       12485,   // finalAmount - totalExpense
  payments: [
    { id:'PAY001', date:'2026-04-22', amount:10000, mode:'cash', notes:'' },
    { id:'PAY002', date:'2026-05-01', amount:5000,  mode:'cash', notes:'' }
  ],
  // amountPaid: 15000  <- computed in-app, not stored
  // balance:    2325   <- computed in-app, not stored
  // paymentStatus: 'partial' <- auto-derived, not stored
  status: 'active'           // 'active' | 'cancelled'
}
```

Collection: rentalEntries (key: 'rl_rentals')

```
{
  id:          'RNT001',
  date:        '2026-04-20',
  farmerId:    'F002',
  machineType: 'tractor',      // tractor | plough_tractor | trailer | water
  unit:        'hour',
  quantity:    3,
  ratePerUnit: 500,
  totalAmount: 1500,
```

```
  payments: [
    { id:'PAY001', date:'2026-04-25', amount:1500, mode:'cash', notes:'full
payment' }
  ],
  // amountPaid: 1500  <- computed
  // balance:    0     <- computed
  // paymentStatus: 'paid' <- auto-derived
  status: 'active'            // 'active' | 'cancelled'
}
```

Collection: driverSalaryEntries (key: 'rl_driver_salary')

```
{
  id:            'DSL001',
  driverId:      'D001',
  date:          '2026-04-20',
  logIds:        ['LOG001'],   // driver logs that contributed to this day
  hoursWorked:   4.25,         // computed from logIds at time of creation
  ratePerHour:   150,          // snapshot from Driver record at time of entry
  salaryEarned:  637.50,       // hoursWorked x ratePerHour
  payments: [
    { id:'PAY001', date:'2026-04-20', amount:400, mode:'cash', notes:'partial' }
  ],
  // amountPaid: 400    <- computed
  // balance:    237.50 <- computed
}
```

Collection: workEntries (key: 'rl_work_entries')

```
{
  id:          'WE001',
  workerId:    'W001',
  date:        '2026-04-20',
  workType:    'Field Levelling',
  baseSalary:  800,
  advance:     200,
  bonus:       100,
  extraAmount: 50,
  // netPayable: 750 <- computed: (800+100+50)-200 = 750
  notes:       ''
}
```

Collection: expenses (key: 'rl_expenses')

```
{
  id:       'EXP001',
  date:     '2026-04-20',
  category: 'food',
  mealTime: 'morning',   // 'morning' | 'afternoon' | 'evening' | null
  amount:   150,
```

```
  source:   'own_farm',  // own_farm | home_expense
  notes:    ''
}
```

Collection: ownFarmIncome (key: 'rl_own_farm_income')

```
{
  id:           'OFI001',
  harvestDate:  '2026-04-20',
  cropType:     'Paddy',
  numberOfBags: 50,
  pricePerBag:  2200,
  totalIncome:  110000,   // numberOfBags x pricePerBag (computed on save)
  source:       'own_farm'
}
```

Collection: lendingRecords (key: 'rl_lending')

```
{
  id:             'LND001',
  personName:     'Selvan',
  village:        'Papanasam',
  amountGiven:    5000,
  dateLent:       '2026-03-15',
  repayments: [
    { id:'RPY001', date:'2026-04-01', amount:2000, notes:'' },
    { id:'RPY002', date:'2026-05-01', amount:3000, notes:'final payment' }
  ],
  // amountReturned: 5000 <- computed from repayments[]
  // status: 'returned'   <- auto-derived
  notes:          ''
}
```

Object: pricingConfig (key: 'rl_pricing_config')

```
{
  harvester: {
    tyre_standard:  2300,   // ₹ per hour — standard field
    tyre_wet_field: 2700,   // ₹ per hour — wet field surcharge
    track:          3000    // ₹ per hour — fixed, editable by Admin only
  },
  rental: {
    tractor:        500,    // ₹ per hour
    plough_tractor: 450,    // ₹ per hour
    trailer:        300     // ₹ per trip
    // water: per-farmer per-season — entered directly in rental entry
  },
  diesel: {
    pricePerLitre: 102      // ₹ — auto-populated in Driver Log diesel input
  },
```

```
  lastUpdated: '2026-04-01'
}
```

> *Pricing Config change rule: Changing a rate in Pricing Config does NOT retroactively change any saved records. All saved Harvester Jobs, Rental Entries, and Driver Logs store a rate snapshot at the time of creation. This ensures historical billing accuracy.*

3.5 Service Layer Design

storage.js

```
export function saveData(key, data) {
  localStorage.setItem(key, JSON.stringify(data));
}
export function getData(key) {
  return JSON.parse(localStorage.getItem(key)) || [];
}
export function getConfig(key) {
  return JSON.parse(localStorage.getItem(key)) || {};
}
export function addRecord(key, record) {
  const list = getData(key);
  list.push(record);
  saveData(key, list);
}
export function updateRecord(key, id, updated) {
  const list = getData(key);
  const idx  = list.findIndex(r => r.id === id);
  if (idx !== -1) { list[idx] = { ...list[idx], ...updated }; saveData(key,
list); }
}
export function deleteRecord(key, id) {
  saveData(key, getData(key).filter(r => r.id !== id));
}
// Add a payment to any payments[] array
export function addPayment(key, recordId, payment) {
  const list   = getData(key);
  const record = list.find(r => r.id === recordId);
  if (record) { record.payments.push(payment); saveData(key, list); }
}
```

calculations.js

```
// Duration with overnight guard
export function getDuration(start, end) {
  const base = '2000-01-01T';
  let s = new Date(base + start);
  let e = new Date(base + end);
  if (e <= s) e = new Date(e.getTime() + 86400000); // +24h
  return (e - s) / 3600000;  // decimal hours
```

```
}

export function getHours(sessions) {
  return sessions.reduce((sum, s) => sum + getDuration(s.start, s.end), 0);
}

export function getDieselCost(mode, value, pricePerLitre) {
  if (mode === 'none') return 0;
  return mode === 'litres' ? value * pricePerLitre : value;
}

export function calcRent(hours, rate, discount = 0) {
  return Math.max(0, (hours * rate) - discount);
}

export function calcNetPay(salary, bonus, extra, advance) {
  return (salary + bonus + extra) - advance;
}

// Driver salary from hours
export function calcDriverSalary(hoursWorked, ratePerHour) {
  return Math.round(hoursWorked * ratePerHour * 100) / 100; // 2dp
}

// Sum a payments[] array
export function sumPayments(payments = []) {
  return payments.reduce((sum, p) => sum + p.amount, 0);
}

// Auto-derive paymentStatus from payments and total
export function getPaymentStatus(totalAmount, payments) {
  const paid    = sumPayments(payments);
  const balance = totalAmount - paid;
  if (paid === 0)      return 'pending';
  if (balance <= 0)    return 'paid';
  return 'partial';
}

export function calcProfit(income, expense) {
  return income - expense;
}
```

billId.js

```
import { getData } from './storage.js';

// Generate next Bill ID for a given season + year
// Format: HB-SSS-YYYY-NNN
export function generateBillId(season, year) {
  const jobs     = getData('rl_harvester_jobs');
  const existing = jobs.filter(j =>
    j.season === season && j.seasonYear === year && j.status !== 'cancelled'
  );
```

```
  const nnn = String(existing.length + 1).padStart(3, '0');
  return `HB-${season}-${year}-${nnn}`;
}

// Season display names
export const SEASON_LABELS = {
  KUR: 'Kuruvai (குறுவை)',
  SAM: 'Samba (சம்பா)',
  THA: 'Thaladi (தலடி)',
};
```

3.6 Backup & Restore with Schema Versioning

```
// Export: add schemaVersion to every backup
export function exportAllData() {
  const backup = {
    schemaVersion:      '2.0.0',
    exportDate:         new Date().toISOString().split('T')[0],
    farmers:            getData('rl_farmers'),
    drivers:            getData('rl_drivers'),
    workers:            getData('rl_workers'),
    driverLogs:         getData('rl_driver_logs'),
    harvesterJobs:      getData('rl_harvester_jobs'),
    rentalEntries:      getData('rl_rentals'),
    driverSalary:       getData('rl_driver_salary'),
    workEntries:        getData('rl_work_entries'),
    expenses:           getData('rl_expenses'),
    ownFarmIncome:      getData('rl_own_farm_income'),
    lendingRecords:     getData('rl_lending'),
    pricingConfig:      getConfig('rl_pricing_config'),
  };
  const blob = new Blob([JSON.stringify(backup, null, 2)],
{type:'application/json'});
  const url  = URL.createObjectURL(blob);
  const a    = document.createElement('a');
  a.href = url;
  a.download = `rl-backup-${backup.exportDate}.json`;
  a.click();
}

// Restore: validate schemaVersion before importing
export function importBackup(jsonString) {
  const data = JSON.parse(jsonString);
  if (!data.schemaVersion) throw new Error('Invalid backup: no schemaVersion');
  if (data.schemaVersion !== '2.0.0') {
    throw new Error(`Incompatible schema: ${data.schemaVersion}. Expected 2.0.0`);
  }
  // Restore all collections
  saveData('rl_farmers',       data.farmers        || []);
  saveData('rl_drivers',       data.drivers        || []);
  // ... (all collections)
  return true;
}
```

3.7 UI Design Patterns

Driver Entry Screen Layout (Mobile)

```
┌──────────────────────────────────────┐
│  ஓட்டுநர் பதிவு  (Driver Entry)    │
├──────────────────────────────────────┤
│  Machine Type (இயந்திர வகை)  [▼]   │
│  Farmer Name  (விவசாயி)       [▼]   │
│  Date         (தேதி)         [today]│
├──────────────────────────────────────┤
│  Work Sessions (வேலை நேரம்)         │
│  [Start] [End]  → Duration: 1.75h   │
│  [Start] [End]  → Duration: 2.50h   │
│  [ + Add Session ]   Total: 4.25h   │
├──────────────────────────────────────┤
│  Diesel (டீசல்)                     │
│  ○ None  ○ Rupees (₹)  ○ Litres     │
│  [Amount]  [Price/L if Litres]       │
│  Total Diesel Cost: ₹ 1,020          │
├──────────────────────────────────────┤
│  [ பதிவு செய்க  —  SAVE LOG ]       │
└──────────────────────────────────────┘
```

Harvester Job Screen Layout (Admin)

```
┌──────────────────────────────────────┐
│  அறுவடை பதிவு  (Harvester Job)      │
│  Bill ID: HB-KUR-2026-001 [auto]    │
├──────────────────────────────────────┤
│  Farmer [▼]  Season [KUR▼] Year[▼] │
│  Machine: ○ Tyre  ○ Track           │
│  Wet Field: ○ Yes  ○ No             │
│  Rate/Hour: ₹ 2300  [editable]      │
├──────────────────────────────────────┤
│  Link Driver Logs:                  │
│  [✓] LOG001 — Apr 20 — 4.25h       │
│  [ ] LOG002 — Apr 21 — 3.50h  (greyed if linked elsewhere)
│  Total Hours: 4.25h                 │
├──────────────────────────────────────┤
│  Gross: ₹9,775  Discount:[  ]       │
│  Final Amount: ₹9,275               │
├──────────────────────────────────────┤
│  Expenses:                          │
│  Diesel from Logs: ₹1,020 [locked]  │
│  Additional Diesel: [   ] ₹          │
│  Worker/Mechanic/Parts: [+ Add]     │
│  Total Expense: ₹2,020              │
│  Net Profit: ₹7,255                 │
├──────────────────────────────────────┤
│  Payments:                          │
│  Apr 22 — ₹5,000 — Cash            │
```

```
│  Balance: ₹4,275  [+ Add Payment]  │
└────────────────────────────────────┘
```

3.8 Data Flow — Complete System

```
DRIVER ENTRY FLOW:
  Driver selects machine, farmer, sessions, diesel
  → getDuration() calculates each session
  → getHours() totals sessions
  → getDieselCost() computes diesel total
  → LOG record saved to rl_driver_logs
  → Admin reviews in Driver Salary tab → salary entry auto-suggested

HARVESTER BILLING FLOW:
  Admin opens new Harvester Job
  → Selects farmer, season (KUR/SAM/THA), year, machine, wet field
  → Links LOG records (filtered by farmer + machine)
  → totalHours auto-computed from linked logs
  → ratePerHour fetched from rl_pricing_config
  → grossAmount, discount, finalAmount computed
  → dieselFromLogs auto-summed from linked logs (read-only)
  → Admin enters additionalDiesel + otherExpenses
  → netProfit computed
  → On save: generateBillId(season, year) → HB-KUR-2026-001
  → HarvesterJob saved to rl_harvester_jobs

PAYMENT RECORDING FLOW:
  Admin taps '+ Add Payment' on any job or rental
  → Enters date, amount, mode (cash / bank / UPI)
  → addPayment() pushes to payments[] array
  → amountPaid, balance, paymentStatus auto-derived on every render
  → No separate payment collection needed

DRIVER SALARY FLOW:
  After driver logs are saved for a day:
  → Admin opens Driver Salary page
  → System shows pending salary entries (days with logs but no salary record)
  → Admin confirms: hoursWorked and ratePerHour snapshot auto-filled
  → salaryEarned computed
  → Admin adds payment(s) to payments[] over time
  → balance tracks running unpaid amount

DASHBOARD FLOW:
  On mount, reads all collections
  → Business income: sum finalAmount from harvesterJobs (active) for month
                   + sum totalAmount from rentalEntries (active) for month
                   + sum totalIncome from ownFarmIncome for month
  → Business expense: sum totalExpense from harvesterJobs for month
                    + sum totalAmount from rentalEntries expenses for month
                    + sum amount from expenses (source=own_farm) for month
  → Home expenses: sum amount from expenses (source=home_expense) — shown
separately
  → Lending: computed from lendingRecords — shown separately
```

```
  → Displays: netProfit, pending bills count, driver salary unpaid total
```

3.9 PWA & Offline Architecture

```
// vite.config.js
import { VitePWA } from 'vite-plugin-pwa';
export default {
  plugins: [VitePWA({
    registerType: 'autoUpdate',
    manifest: {
      name:             'கிராம கணக்கு',
      short_name:       'கணக்கு',
      theme_color:      '#1B3A6B',
      background_color: '#FFFFFF',
      display:          'standalone',
      icons: [{ src:'/icon-192.png', sizes:'192x192', type:'image/png' }]
    }
  })]
};
```

PHASE 4 Development Execution Plan

4.1 Development Strategy

Module-by-module progression. No module is started until the previous module is functionally complete and tested locally. Development flow for every module: UI → State → Logic → Save → Test. Pricing Config is initialised on Day 1 because every other module depends on it for rates.

4.2 Priority Order

Priority	Module	Reason
1	Project Setup + Pricing Config init	Foundation; every module needs pricing data
2	Driver Entry UI (no logic)	Visual structure before wiring
3	Driver Logic + Time Calculation	Core data feeding harvester + salary
4	Diesel Logic + LocalStorage Save	Completes Driver Logs module
5	Harvester Management + Bill ID	Core revenue; links to driver logs
6	Machinery Rental + Payment History	Secondary revenue with payments[] pattern
7	Worker Management + Driver Salary Ledger	Expense tracking + driver pay automation
8	Dashboard	Aggregation of all completed modules
9	Reports + PWA Setup	All 5 report types + offline install
10	Export/Import + Final Testing	Backup with schemaVersion + end-to-end UAT

4.3 Day-wise Execution Plan

Day 1 — Project Setup, Base Components, Pricing Config

Goal: Running app with correct structure, reusable components, and a live pricing config that all modules will read from.

23. Create project: npm create vite@latest rural-ledger --template react
24. Remove all boilerplate; set up full folder structure from Section 3.3
25. Install: npm install react-router-dom
26. Create common components: Button.jsx, InputField.jsx, TamilLabel.jsx, SelectField.jsx
27. Create storage.js with all 6 functions (saveData, getData, getConfig, addRecord, updateRecord, addPayment)
28. Create initialData.js: F000 Own Farm, 2 sample drivers with salaryRatePerHour, default pricingConfig
29. Create PricingConfig.jsx page — simple form showing all rates, Admin-editable, saves to rl_pricing_config
30. Create idGenerator.js with auto-ID functions for all entity types
31. Wire basic routing: Login → Dashboard → DriverEntry → PricingConfig
32. Verify: app runs, pricing config screen saves and reads correctly

Day 2 — Driver Entry UI (No Logic)

Goal: The Driver Entry form is visually complete on mobile. All fields visible, Tamil labels on every field. Session add/remove works in state. No calculations, no saving.

33. Machine type dropdown (5 options with Tamil labels from machineTypes.js)
34. Farmer name dropdown (from initialData)
35. Date picker (defaults to today)
36. Sessions section: start + end time inputs per session row
37. + Add Session button appends empty session; X button removes
38. Diesel section: radio toggle — None / Rupees / Litres
39. Conditional price-per-litre input (visible only in Litres mode)
40. Diesel total display (₹0 at this stage)
41. SAVE button — console.log only
42. Verify form renders correctly on 375px mobile viewport

Day 3 — Driver Logic + Time Calculation

Goal: All session calculations work. Validation errors shown.

43. Implement getDuration() in timeUtils.js with overnight guard
44. Implement getHours() in calculations.js
45. Wire each session's time inputs to update durationHours on blur
46. Display running total hours below session list
47. Validation: end time after start time — inline error per row
48. Test: single session, multiple sessions, midnight-crossing session

Day 4 — Diesel Logic + Data Save

Goal: Diesel modes work, completed logs save to LocalStorage and persist on reload.

49. Implement getDieselCost() in calculations.js — all 3 modes
50. Wire Litres mode to auto-populate price from pricingConfig.diesel.pricePerLitre

51. Display computed diesel total below diesel section
52. Wire SAVE: validate all fields → compute totals → addRecord('rl_driver_logs', log)
53. On save: clear form, show Tamil success toast
54. Show today's log list below form
55. Verify: save → close tab → reopen → data present

Day 5 — Harvester Management + Bill ID

Goal: Admin creates harvester jobs with auto-calculated billing, linked logs, and generated Bill IDs.

56. Create Harvester.jsx and billId.js
57. Create seasons.js enum: KUR/SAM/THA with Tamil display names
58. Farmer selector + season dropdown + year + machine + wet field toggle
59. Rate auto-fetched from pricingConfig based on machineType + wetField
60. LogLinker.jsx: filter logs by farmerId + machineType; checkbox list with already-linked guard
61. totalHours auto-computed on checkbox change
62. grossAmount, discount input, finalAmount display
63. dieselFromLogs auto-summed from selected logs (read-only display)
64. additionalDiesel input + otherExpenses dynamic rows
65. netProfit display
66. On save: generateBillId(season, year) → assign to record → save to rl_harvester_jobs
67. PaymentHistory.jsx: show payments[] with + Add Payment button
68. Verify: Bill IDs increment correctly within a season (HB-KUR-2026-001, -002...)

Day 6 — Machinery Rental + Payment History Pattern

Goal: All rental types work with payments[] array. Payment history reusable component proven.

69. Create Rental.jsx
70. Machine selector auto-sets unit type (tractor → hour, trailer → trip)
71. Rate fetched from pricingConfig.rental per machine type
72. totalAmount = quantity × ratePerUnit displayed
73. Reuse PaymentHistory.jsx for rental payments — confirm it works for both modules
74. sumPayments(), getPaymentStatus() functions tested
75. Rental list view with farmer name, machine, amount, paymentStatus badge
76. Verify: add payment → amountPaid and balance update correctly

Day 7 — Worker Management + Driver Salary Ledger

Goal: Worker net payable works. Driver salary is auto-calculated from existing logs.

77. Worker list page with + Add Entry per worker
78. calcNetPay() wired live — shows warning if negative
79. Save to rl_work_entries

80. Create DriverSalary.jsx
81. On load: for each driver, find days with logs that have no salary entry yet — show as 'pending salary'
82. Admin taps a pending day: hoursWorked pre-filled from logs, ratePerHour pre-filled from driver record
83. calcDriverSalary() computes salaryEarned
84. Admin confirms → DSL entry saved to rl_driver_salary
85. Reuse PaymentHistory.jsx for salary payments
86. Driver view: when driver logs in, DriverSalary page shows their own ledger (read-only)

Day 8 — Dashboard

Goal: Dashboard shows accurate live numbers from all 12 modules.

87. Read all collections on mount
88. Month selector: default current month
89. Compute income, business expense, net profit using source tags and date filtering
90. Show: pending bills count, total pending balance, driver salary unpaid total
91. Home expenses and lending shown as separate awareness panels (not in profit)
92. Colour coding: green for profit, red for loss

Day 9 — Reports + PWA Setup

Goal: All 5 report types functional. App is installable and offline-capable.

93. Report 1: Monthly Summary — income/expense/profit breakdown by source, filter by month/year
94. Report 2: Machine-wise — group by machineType, revenue/expense/profit per machine
95. Report 3: Farmer Balance — outstanding per farmer, sorted by balance
96. Report 4: Driver Salary — per driver per month; earned, paid, balance
97. Report 5: Season Harvest — all jobs in a season with Bill IDs and payment status
98. Install vite-plugin-pwa; configure manifest.json with Tamil app name
99. Test: install on Android Chrome → Add to Home Screen
100. Test: WiFi off → open app → all data accessible → add new entry → data saved

Day 10 — Export, Import, PDF, Final Testing

Goal: Backup/restore with schema validation works. Full end-to-end flow verified.

101. exportAllData() with schemaVersion: '2.0.0' — verified JSON downloads
102. importBackup() with schemaVersion validation — tested with correct and wrong version
103. PDF report: jsPDF monthly summary — Bill IDs included
104. WhatsApp share: text summary of monthly report via wa.me
105. End-to-end test: Driver log → Harvester job → Bill ID → Payment → Dashboard
106. Usability test with real driver user; fix any confusion
107. Tamil label audit: every input field has correct Tamil spelling
108. Final: clean console errors, remove all console.log statements

PHASE 5 Testing & Quality Assurance Plan

5.1 Testing Objectives

Verify that every module is correct for real-world use, no data is lost or corrupted, all calculations produce exact results, Bill IDs are unique and correctly formatted, payment history is accurate across partial payments, driver salary matches machine hours, and the interface is usable by non-technical Tamil-speaking users without assistance.

5.2 Unit Test Cases — Calculations

Function	Input	Expected	Edge Case?
getDuration()	'09:00', '10:45'	1.75 hours	No
getDuration()	'23:00', '01:00'	2.00 hours	YES — overnight
getDuration()	'00:00', '00:00'	0 (validation error)	YES — same time
getHours()	[1.75h, 2.50h]	4.25 hours	No
getDieselCost()	rupees, 2000	2000	No
getDieselCost()	litres, 10, 102	1020	No
getDieselCost()	none, 0, 0	0	No
calcRent()	4.25, 2300, 500	9275	No
calcRent()	0, 2300, 0	0	YES — zero hours
calcNetPay()	800, 100, 50, 200	750	No
calcNetPay()	500, 0, 0, 700	-200 (warning shown)	YES — over-advance
calcDriverSalary()	4.25, 150	637.50	No
calcDriverSalary()	0, 150	0	YES — no hours
sumPayments()	[{amount:3000}, {amount:5000}]	8000	No
sumPayments()	[]	0	YES — no payments
getPaymentStatus()	10000, []	'pending'	No
getPaymentStatus()	10000, [{amount:5000}]	'partial'	No
getPaymentStatus()	10000, [{amount:10000}]	'paid'	No

5.3 Bill ID Test Cases

Scenario	Input	Expected Bill ID	Verification
First KUR 2026 job	season=KUR, year=2026, 0 existing KUR-2026 jobs	HB-KUR-2026-001	Counter starts at 001
Second KUR 2026 job	season=KUR, year=2026, 1 existing KUR-2026 job	HB-KUR-2026-002	Increments correctly
First SAM 2026 job	season=SAM, year=2026, 0 SAM-2026 jobs	HB-SAM-2026-001	Season isolation — does not mix with KUR
After cancellation	KUR-2026 has 3 active + 1 cancelled	HB-KUR-2026-005	Cancelled jobs not counted; NNN=5
New year	season=KUR, year=2027	HB-KUR-2027-001	Year resets the counter

5.4 Integration Test Scenarios

Scenario 1 — Full Billing Flow with Payment History

```
1. Driver Kumar logs sessions for farmer Ravi (F001), harvester_tyre:
   Session: 09:00-10:45 (1.75h), 11:30-14:00 (2.50h)
   Diesel: 10 litres × ₹102 = ₹1020
   → LOG001 saved

2. Admin creates Harvester Job:
   Farmer: Ravi, Season: KUR, Year: 2026, Machine: tyre, Wet Field: OFF
   Links LOG001 → totalHours: 4.25, Rate: ₹2300
   Gross: 4.25 × 2300 = ₹9,775
   Discount: ₹500 → Final: ₹9,275
   dieselFromLogs: ₹1020 (read-only)
   additionalDiesel: ₹0, Worker: ₹1500
   totalExpense: ₹2520, netProfit: ₹6,755
   → On save: billId = HB-KUR-2026-001

3. Admin records payments:
   Apr 22: ₹5,000 — payments[0]
   May 01: ₹4,275 — payments[1]
   amountPaid: ₹9,275, balance: ₹0, status: 'paid'

4. Farmer Ravi opens Farmer View:
   Sees: HB-KUR-2026-001 — ₹9,275
   Payment history: Apr 22 ₹5,000 | May 01 ₹4,275
   Balance: ₹0 — Fully paid

5. Dashboard for April 2026:
   Income includes ₹9,275 from this job
   Expense includes ₹2,520
   Net profit: ₹6,755 reflected correctly
```

Scenario 2 — Driver Salary from Logs

```
1. Driver Kumar (salaryRatePerHour: ₹150) logs on April 20:
   LOG001: 4.25 hours (Ravi's farm)
   LOG002: 2.00 hours (Own Farm F000)
   Total hours on Apr 20: 6.25h

2. Admin opens Driver Salary page → sees Apr 20 pending for Kumar
   hoursWorked: 6.25 (pre-filled), ratePerHour: ₹150 (pre-filled)
   salaryEarned: ₹937.50

3. Admin pays ₹600 on Apr 20 → payments[0] = {date:Apr 20, amount:600}
   balance: ₹337.50

4. Admin pays ₹337.50 on Apr 25 → payments[1]
   balance: ₹0 — fully paid

5. Driver Kumar logs in, views Salary Ledger:
   Apr 20: 6.25h × ₹150 = ₹937.50 | Paid: ₹937.50 | Balance: ₹0
```

Scenario 3 — Diesel Double-Counting Prevention

```
1. Driver logs 10L diesel in LOG001 → diesel.total: ₹1020

2. Admin creates Harvester Job, links LOG001:
   dieselFromLogs: ₹1020 shown as read-only
   additionalDiesel: ₹0
   totalDiesel: ₹1020

3. WRONG attempt: Admin also enters ₹1020 in additionalDiesel
   totalDiesel would show ₹2040 — INCORRECT
   System does not prevent this automatically, but the UI shows
   'Diesel from Logs: ₹1020 (already included)' as a warning label
   next to the additionalDiesel input.

4. CORRECT use of additionalDiesel:
   Admin bought 5L bulk diesel for transport not logged by driver
   additionalDiesel: ₹510
   totalDiesel: ₹1020 + ₹510 = ₹1530 — CORRECT
```

5.5 Offline Testing Protocol

Test	Steps	Expected Result
Cold offline start	Open app, disable WiFi, reload page	App loads from PWA cache; all data visible
Offline data entry	WiFi off → add driver log → close tab	Log saved in LocalStorage; visible on reopen
Bill ID generation	WiFi off → create Harvester	Bill ID generated correctly from local data

offline	Job	
PWA install	Chrome menu → Add to Home Screen	App opens standalone, Tamil name shown
Backup export offline	Click Export Data with WiFi off	JSON file downloads with schemaVersion: 2.0.0
Backup restore	Import valid 2.0.0 JSON	All collections restored; pricing config restored
Wrong version restore	Import a 1.0.0 JSON backup	Error shown: 'Incompatible schema: 1.0.0'

5.6 Acceptance Criteria Checklist

#	Criteria	Pass Condition
1	Driver log entry	Completed by driver in < 2 minutes without help
2	Time calculation	All unit tests in 5.2 produce exact output
3	Overnight session	23:00 to 01:00 = 2.00 hours correctly
4	Harvester billing	Rent auto-calculated; Bill ID generated in HB-SSS-YYYY-NNN format
5	Bill ID uniqueness	No two active jobs share the same Bill ID
6	Payment history	Multiple payments recorded with dates; amountPaid and balance auto-computed
7	Driver salary	Salary auto-calculated from log hours; partial payments tracked
8	Diesel no double-count	dieselFromLogs shown as read-only; additionalDiesel is separate
9	Edit/delete guards	Cannot delete a linked log; cannot hard-delete a job with payments
10	Offline persistence	All data survives app close and reopen with WiFi off
11	Dashboard accuracy	Monthly profit matches sum of all individual entries
12	Farmer view	Farmer sees bill IDs, payment history, balance — no other farmer's data
13	Backup restore	Export → clear → import → all data recovered; wrong schema blocked
14	PWA install	App installable, runs offline from home screen
15	Tamil labels	Every input field has a visible, correctly-spelled Tamil label

PHASE 6 Deployment & Delivery Plan

6.1 Deployment Strategy

Static web application hosted on Vercel. No server, no database — the deployment is a simple static file upload. Vercel provides automatic HTTPS, global CDN, and instant deployment from GitHub. The PWA service worker ensures the app is cached after the first load and runs fully offline thereafter.

> *Why Vercel: No Play Store delay. Any update is live in under 60 seconds. Works on any Android browser. HTTPS automatic. Free tier sufficient for this scale.*

6.2 Build & Deployment Steps

Step 1 — Production Build

```
npm run build

# Verify output:
# dist/index.html
# dist/assets/  (hashed JS and CSS)
# dist/manifest.json  (PWA)

# Local preview before deploying:
npm run preview
```

Step 2 — Push to GitHub

```
git init
git add .
git commit -m 'feat: rural ledger v2.0 — full sdlc implementation'
git remote add origin https://github.com/YOUR_USERNAME/rural-ledger.git
git push -u origin main
```

Step 3 — Deploy to Vercel

109. Go to vercel.com → Sign in with GitHub
110. New Project → Select rural-ledger repository
111. Framework: Vite (auto-detected), Build: npm run build, Output: dist
112. Deploy → Live URL ready in ~90 seconds

6.3 Mobile Installation (PWA)

113. Open Vercel URL in Chrome on Android
114. Tap three-dot menu (⋮) → 'Add to Home Screen'
115. Name shows as 'கிராம கணக்கு' — tap Add
116. Turn off WiFi. Open app from home screen. Verify it loads and all data accessible.

> *After installation: fullscreen (no address bar), Tamil app name, complete offline capability. Behaves like a native app.*

6.4 Data Backup & Recovery Strategy

Backup Type	Frequency	Method	Recovery Method
Weekly JSON Backup	Every Sunday	Settings → Backup Data → download JSON	Upload JSON → Restore → schemaVersion validated
Pre-Update Backup	Before any app change	Same as above	Same as above
Monthly PDF Archive	First of each month	Reports → Monthly → Export PDF	Reference only — not restorable

6.5 Client Handover Package

- Live application URL (Vercel link)
- Tamil user guide document (printed + digital)
- Admin setup guide: pricing config, adding farmers/drivers, setting seasons
- Backup and restore instructions with screenshots
- Bill ID reference card (explain HB-KUR/SAM/THA-YYYY-NNN format)
- Driver salary setup guide: entering ratePerHour, confirming daily entries
- Developer contact for 30-day post-delivery support period

Tamil User Guide Extract — Driver Entry

```
Driver Entry (ஓட்டுநர் பதிவு) — படிப்படியாக:

1.  இயந்திரம் தேர்வு செய்யவும்
    (Select the machine type)

2.  விவசாயி பெயர் தேர்வு செய்யவும்
    (Select the farmer's name)

3.  தொடக்க நேரம் பதிவு செய்யவும்
    (Enter the start time)

4.  முடிவு நேரம் பதிவு செய்யவும்
```

```
    (Enter the end time — system calculates hours automatically)

5.  மேலும் நேரம் சேர்க்க '+' அழுத்தவும்
    (Tap '+' to add another work session if needed)

6.  டீசல் முறை தேர்வு செய்யவும்
    (Choose diesel mode: None / Rupees / Litres)

7.  டீசல் தொகை அல்லது லிட்டர் பதிவு செய்யவும்
    (Enter diesel amount — price per litre is auto-filled)

8.  'பதிவு செய்க' அழுத்தவும்
    (Tap SAVE — entry is saved to device immediately)
```

Tamil User Guide Extract — Checking Bill ID

```
உங்கள் பில் எண் (Bill ID) பார்க்க:

1. விவசாயி பதிவு (Farmer View) திரை திறக்கவும்
2. உங்கள் பெயர் தேர்வு செய்யவும்
3. பில் பட்டியல் தோன்றும்:
   உதாரணம்: HB-KUR-2026-001
   HB = அறுவடை பில்
   KUR = குறுவை பருவம்
   2026 = ஆண்டு
   001 = பில் எண்
4. ஒவ்வொரு பிலிலும் செலுத்திய தொகை மற்றும் நிலுவை தெரியும்
```

6.6 Post-Deployment Risks & Mitigations

Risk	Likelihood	Mitigation
Browser storage cleared	Medium	Weekly JSON backup habit; trained at handover
Wrong season selected for Bill ID	Medium	Season dropdown with Tamil names; confirm on save screen
Driver forgets to log diesel	Medium	Harvester UI shows dieselFromLogs as ₹0 — admin notices and adds additionalDiesel
Admin changes rate mid-season	Low	Pricing Config change does not affect saved jobs; only new jobs use new rate
Device upgrade / data migration	Low	JSON backup portable; restore validated by schemaVersion
App not loading (Vercel outage)	Very Low	PWA cache serves app offline; Vercel SLA 99.99%

6.7 Future Phase Roadmap

Feature	Phase	Dependency
Cloud sync across devices	Phase 2	Backend API + database
Multi-device real-time sync	Phase 2	Cloud sync first
OTP-based admin login	Phase 2	Backend auth service
Farmer WhatsApp bill notifications	Phase 2	WhatsApp Business API
Multi-entrepreneur / multi-tenant	Phase 3	Full multi-tenant architecture
Advanced analytics and charts	Phase 3	3+ months data accumulation
Bank account reconciliation	Phase 3	Banking API integration

6.8 Final Delivery Checklist

The project is considered COMPLETE only when every item below is verified by live demonstration with the client.

#	Item	Verified By
1	App deployed and accessible via Vercel URL	Admin opens URL on phone
2	App installed as PWA on client's Android phone	Icon visible on home screen; Tamil name shown
3	App loads and works with WiFi off	Live demo without internet
4	Driver logs an entry without assistance	Driver usability test < 2 minutes
5	Harvester job created with correct Bill ID (HB-KUR/SAM/THA-YYYY-NNN)	Test case verified
6	Multiple payments recorded for a bill; balance auto-updates	Live demo
7	Driver salary auto-calculated from log hours	Live demo with test data
8	Dashboard shows correct income/expense/profit for current month	Data cross-checked
9	Farmer sees bills, Bill IDs, and payment history (own data only)	Farmer demo
10	JSON backup exports with schemaVersion: 2.0.0	Export and inspect file
11	JSON restore works; wrong version blocked with clear error	Import old + new version
12	PDF report generated with Bill IDs	Printed or screenshot
13	All Tamil labels visible and correctly spelled	Admin review

14	Printed Tamil user guide handed to client	Physical handover
15	Client confirms all features demonstrated and understood	Verbal sign-off

---

END OF SDLC DOCUMENTATION — VERSION 2.0

Rural Ledger & Machinery Management System | Revised Final | April 2026

All 12 modules defined | All critical issues resolved | Bill ID: HB-SSS-YYYY-NNN